

```

75         "tags": {"contains": "enterprise"}
76     }
77
78     Returns:
79         List of matching contacts
80     """
81
82     # Build WHERE clause dynamically
83     where_clauses = ["user_id = $1", "is_deleted = false"]
84     params = [user_id]
85     param_idx = 2
86
87     for field, condition in filters.items():
88         clause, param_idx = self._build_filter_clause(
89             field, condition, params, param_idx
90         )
91         if clause:
92             where_clauses.append(clause)
93
94     # Build ORDER BY
95     order_clause = self._build_order_clause(sort_by)
96
97     # Execute query
98     query = f"""
99         SELECT
100             id, email, first_name, last_name,
101             company, job_title, profile
102         FROM contacts
103         WHERE {' AND '.join(where_clauses)}
104         {order_clause}
105         LIMIT ${param_idx}
106     """
107     params.append(limit)
108
109     rows = await self.db.fetch(query, *params)
110     return [dict(row) for row in rows]
111
112     def _build_filter_clause(
113         self,
114         field: str,
115         condition: Any,
116         params: List,
117         param_idx: int
118     ) -> tuple[str, int]:
119         """
120         Build SQL WHERE clause for JSONB field
121
122         Supports operators: =, >, >=, <, <=, contains, length
123         """
124
125         # Simple fields (not in profile JSONB)
126         if field in ["email", "company", "job_title",
127 "do_not_contact"]:
128             params.append(condition)
129             return f"{field} = ${param_idx}", param_idx + 1
130
131         # Tags (array contains)
132         if field == "tags":
133             if isinstance(condition, dict) and "contains" in

```

```

134         return f"tags @> ARRAY[${param_idx}>::varchar[]",
param_idx + 1
135
136     # Profile JSONB paths
137     if "." not in field:
138         # Top-level profile field
139         path = field
140     else:
141         # Nested path: "relationship.strength_score"
142         path = field
143
144     # Handle different condition types
145     if isinstance(condition, dict):
146         # Operator-based: {">=": 70}
147         for op, value in condition.items():
148             if op in [ ">=", ">", "<=", "<", "=" ]:
149                 params.append(value)
150                 return (
151                     f"(profile#>>'{{{path}}}'::text[])::int
{op} ${param_idx}",
152                     param_idx + 1
153                 )
154             elif op == "contains":
155                 params.append(f"%{value}%")
156                 return (
157                     f"profile#>>'{{{path}}}'::text[] ILIKE
${param_idx}",
158                     param_idx + 1
159                 )
160             elif op == "length":
161                 # Array length: {"length": {">": 2}}
162                 length_op, length_val = list(value.items())[0]
163                 params.append(length_val)
164                 return (
165                     f"jsonb_array_length(profile#>>'{{{path}}}'::text[]) {length_op}
${param_idx}",
166                     param_idx + 1
167                 )
168     else:
169         # Exact match
170         params.append(str(condition))
171         return (
172             f"profile#>>'{{{path}}}'::text[] = ${param_idx}",
173             param_idx + 1
174         )
175
176     return None, param_idx
177
178     def _build_order_clause(self, sort_by: Optional[str]) -> str:
179         """
180         Build ORDER BY clause
181         """
182         if not sort_by:
183             return ""
184
185         sort_mapping = {
186             "relationship_strength": "
(profile#>>'{relationship,strength_score}'::int DESC",
187             "priority_score": "
(profile#>>'{segmentation,priority_score}'::int DESC",

```

```

188         "last_interaction": "
(profile#>>'{relationship,last_interaction_at}'):timestamp DESC",
189         "created_at": "created_at DESC"
190     }
191
192     return f"ORDER BY {sort_mapping.get(sort_by, 'created_at
DESC')}"
193
194     @tool
195     async def search_contacts_semantic(
196         self,
197         user_id: str,
198         query: str,
199         filters: Optional[Dict] = None,
200         top_k: int = 10
201     ) -> List[Dict]:
202         """
203         Semantic search using vector embeddings in Redis
204
205         Args:
206             user_id: User ID
207             query: Natural language query
208             filters: Optional filters (lifecycle_stage, min_score,
etc.)
209             top_k: Number of results
210
211         Returns:
212             List of contacts with similarity scores
213         """
214         from redis.commands.search.query import Query
215         import numpy as np
216
217         # Generate embedding for query
218         embedding = await self._generate_embedding(query)
219
220         # Build Redis query
221         filter_expr = f"@user_id:{user_id}"
222
223         if filters:
224             if "lifecycle_stage" in filters:
225                 filter_expr += f" @lifecycle_stage:
{{{filters['lifecycle_stage']}}}"
226             if "min_relationship_strength" in filters:
227                 filter_expr += f" @relationship_strength:
[{filters['min_relationship_strength']} +inf]"
228
229         search_query = (
230             Query(f"({filter_expr}) => [KNN {top_k} @embedding
$vec AS score]")
231             .sort_by("score")
232             .return_fields("contact_id", "name", "company",
"score")
233             .dialect(2)
234         )
235
236         # Execute search
237         results = self.redis.ft("idx:contacts").search(
238             search_query,
239             query_params={"vec":
np.array(embedding).astype(np.float32).tobytes()}
240         )
241

```

```

242         # Fetch full contact data from PostgreSQL
243         contacts = []
244         for doc in results.docs:
245             contact = await self.get_contact(doc.contact_id)
246             contact["similarity_score"] = 1 - float(doc.score)
247             contacts.append(contact)
248
249         return contacts
250
251     async def _generate_embedding(self, text: str) -> List[float]:
252         """
253         Generate embedding using OpenAI or Anthropic
254         """
255         # Implementation depends on embedding provider
256         # Example: OpenAI text-embedding-3-small
257         import openai
258
259         response = await openai.Embedding.acreate(
260             model="text-embedding-3-small",
261             input=text
262         )
263
264         return response["data"][0]["embedding"]

```

Usage in Agent:

Code block

```

1  # In any agent
2  contact = await self.use_skill("contact_query", "get_contact",
3                                  contact_id="uuid")
4
5  # Filter contacts
6  prospects = await self.use_skill("contact_query",
7                                  "filter_contacts",
8                                  user_id="user-uuid",
9                                  filters={
10                                     "lifecycle_stage": "Opportunity",
11                                     "relationship_strength_score": {">=": 70},
12                                     "ai_insights.buying_signals": {"length": {">": 2}}
13                                 },
14                                  sort_by="priority_score",
15                                  limit=50
16          )
17
18  # Semantic search
19  similar = await self.use_skill("contact_query",
20                                  "search_contacts_semantic",
21                                  user_id="user-uuid",
22                                  query="VP Engineering interested in automation",
23                                  top_k=10
24          )

```

2.2 Skill: SegmentationLogicSkill

Purpose: Apply user-defined segmentation criteria and scoring rules

File: `skills/segmentation_logic_skill.py`

```

1 from agent_sdk import Skill, tool
2 from typing import Dict, List, Any
3
4 class SegmentationLogicSkill(Skill):
5     """
6     Apply user-defined segmentation logic
7
8     Supports:
9     - Custom filter criteria
10    - Custom scoring rules
11    - User-defined ICP matching
12    - Dynamic segment membership
13    """
14
15    name = "segmentation_logic"
16    description = "Apply user-defined segmentation criteria"
17
18    def __init__(self, db_pool):
19        self.db = db_pool
20
21    @tool
22    async def evaluate_contact_fit(
23        self,
24        contact: Dict,
25        segmentation_id: str
26    ) -> Dict:
27        """
28        Evaluate contact against segment criteria
29
30        Args:
31            contact: Contact dict with profile
32            segmentation_id: Segment to evaluate against
33
34        Returns:
35            {
36                "fit_score": 85,
37                "score_breakdown": {...},
38                "matched_criteria": [...],
39                "failed_criteria": [...]
40            }
41        """
42
43        # Fetch segmentation
44        segment = await self.db.fetchrow(
45            "SELECT * FROM segmentations WHERE id = $1",
46            segmentation_id
47        )
48
49        if not segment:
50            raise ValueError(f"Segmentation {segmentation_id} not found")
51
52        criteria = segment["criteria"]
53        icp_definition = segment["icp_definition"]
54
55        # Evaluate filters (boolean pass/fail)
56        filter_results = await self._evaluate_filters(contact,
57            criteria.get("filters", []))
58
59        # Calculate scoring components
60        score_breakdown = {}

```

```

61         # Standard scoring components
62         score_breakdown["title_match"] = self._score_title_match(
63             contact["job_title"],
64             icp_definition.get("ideal_titles", [])
65         )
66
67         score_breakdown["company_size_match"] =
self._score_company_size(
68             contact,
69             icp_definition.get("ideal_company_size", [])
70         )
71
72         score_breakdown["industry_match"] = self._score_industry(
73             contact,
74             icp_definition.get("ideal_industries", [])
75         )
76
77         score_breakdown["engagement_level"] =
contact.get("profile", {}).get(
78             "relationship", {}
79         ).get("strength_score", 0)
80
81         # User-defined scoring rules
82         user_score = await self._apply_user_scoring_rules(
83             contact,
84             criteria.get("scoring_rules", [])
85         )
86         score_breakdown["user_defined_score"] = user_score
87
88         # Aggregate score (weighted average)
89         fit_score = self._calculate_weighted_score(score_breakdown)
90
91         return {
92             "fit_score": fit_score,
93             "score_breakdown": score_breakdown,
94             "matched_criteria": filter_results["matched"],
95             "failed_criteria": filter_results["failed"],
96             "passes_filters": filter_results["passes_all"]
97         }
98
99     async def _evaluate_filters(
100         self,
101         contact: Dict,
102         filters: List[Dict]
103     ) -> Dict:
104         """
105         Evaluate filter criteria (user-defined logic)
106
107         Filter format:
108         {
109             "field":
"profile.confirmed_facts.budget_timeline.budget_available",
110             "operator": ">=",
111             "value": 50000,
112             "user_defined": true,
113             "reason": "Only target contacts with sufficient budget"
114         }
115         """
116
117         matched = []
118         failed = []
119

```

```

120         for filter_rule in filters:
121             field = filter_rule["field"]
122             operator = filter_rule["operator"]
123             expected_value = filter_rule["value"]
124
125             # Extract field value from contact
126             actual_value = self._extract_nested_field(contact,
127 field)
128
129             # Apply operator
130             passes = self._apply_operator(actual_value, operator,
131 expected_value)
132
133             if passes:
134                 matched.append(filter_rule)
135             else:
136                 failed.append({
137                     **filter_rule,
138                     "actual_value": actual_value
139                 })
140
141             return {
142                 "matched": matched,
143                 "failed": failed,
144                 "passes_all": len(failed) == 0
145             }
146
147     def _extract_nested_field(self, obj: Dict, field_path: str) ->
148 Any:
149         """
150         Extract nested field value
151
152         Examples:
153         "job_title" -> contact["job_title"]
154         "profile.relationship.strength_score" ->
155 contact["profile"]["relationship"]["strength_score"]
156         """
157         parts = field_path.split(".")
158         value = obj
159
160         for part in parts:
161             if isinstance(value, dict):
162                 value = value.get(part)
163             else:
164                 return None
165
166         return value
167
168     def _apply_operator(self, actual: Any, operator: str,
169 expected: Any) -> bool:
170         """
171         Apply comparison operator
172         """
173         if actual is None:
174             return False
175
176         operators = {
177             "==": lambda a, e: a == e,
178             "!=": lambda a, e: a != e,
179             ">": lambda a, e: a > e,
180             ">=": lambda a, e: a >= e,
181             "<": lambda a, e: a < e,

```

```

177         "<=": lambda a, e: a <= e,
178         "contains": lambda a, e: e in str(a),
179         "in": lambda a, e: a in e,
180         "not_in": lambda a, e: a not in e
181     }
182
183     return operators.get(operator, lambda a, e: False)(actual,
expected)
184
185     def _score_title_match(self, title: str, ideal_titles:
List[str]) -> int:
186         """
187         Score title match (0-100)
188         """
189         if not title or not ideal_titles:
190             return 50
191
192         title_lower = title.lower()
193
194         # Exact match
195         if any(ideal.lower() == title_lower for ideal in
ideal_titles):
196             return 100
197
198         # Partial match
199         if any(ideal.lower() in title_lower for ideal in
ideal_titles):
200             return 80
201
202         # Seniority match
203         seniority = ["vp", "director", "head", "chief", "cto",
"ceo", "president"]
204         if any(s in title_lower for s in seniority):
205             return 70
206
207         return 30
208
209     def _score_company_size(self, contact: Dict, ideal_sizes:
List[str]) -> int:
210         """
211         Score company size match (0-100)
212         """
213         company_size = contact.get("profile",
{}).get("company_data", {}).get("company_size")
214
215         if not company_size or not ideal_sizes:
216             return 50
217
218         if company_size in ideal_sizes:
219             return 100
220
221         # Adjacent size scoring
222         size_order = ["1-10", "11-50", "51-200", "201-1000",
"1000+"]
223         try:
224             contact_idx = size_order.index(company_size)
225             ideal_indices = [size_order.index(s) for s in
ideal_sizes if s in size_order]
226             min_distance = min(abs(contact_idx - idx) for idx in
ideal_indices)
227
228             return max(0, 100 - (min_distance * 30))

```

```

229         except (ValueError, IndexError):
230             return 50
231
232     def _score_industry(self, contact: Dict, ideal_industries:
List[str]) -> int:
233         """
234         Score industry match (0-100)
235         """
236         industry = contact.get("profile", {}).get("company_data",
        {}).get("industry")
237
238         if not industry or not ideal_industries:
239             return 50
240
241         if industry in ideal_industries:
242             return 100
243
244         # Partial match
245         industry_lower = industry.lower()
246         if any(ideal.lower() in industry_lower for ideal in
ideal_industries):
247             return 70
248
249         return 20
250
251     async def _apply_user_scoring_rules(
252         self,
253         contact: Dict,
254         scoring_rules: List[Dict]
255     ) -> int:
256         """
257         Apply user-defined scoring rules
258
259         Rule format:
260         {
261             "rule_id": "budget_check",
262             "field":
"profile.confirmed_facts.budget_timeline.budget_available",
263             "operator": ">=",
264             "value": 50000,
265             "points": 25,
266             "user_defined": true,
267             "reason": "Minimum budget requirement"
268         }
269         """
270         total_points = 0
271         max_points = sum(rule.get("points", 0) for rule in
scoring_rules)
272
273         for rule in scoring_rules:
274             field_value = self._extract_nested_field(contact,
rule["field"])
275
276             if self._apply_operator(field_value, rule["operator"],
rule["value"]):
277                 total_points += rule.get("points", 0)
278
279         # Normalize to 0-100
280         if max_points > 0:
281             return int((total_points / max_points) * 100)
282         return 0
283

```

```

284     def _calculate_weighted_score(self, score_breakdown: Dict) ->
285         int:
286             """
287             Calculate weighted average of score components
288             """
289             weights = {
290                 "title_match": 0.25,
291                 "company_size_match": 0.15,
292                 "industry_match": 0.15,
293                 "engagement_level": 0.30,
294                 "user_defined_score": 0.15
295             }
296
297             total_score = sum(
298                 score_breakdown.get(component, 0) * weight
299                 for component, weight in weights.items()
300             )
301
302             return int(total_score)
303
304     @tool
305     async def update_segmentation_criteria(
306         self,
307         segmentation_id: str,
308         user_id: str,
309         criteria_updates: Dict
310     ) -> Dict:
311         """
312         Update segmentation criteria with user input
313
314         Args:
315             segmentation_id: Segment to update
316             user_id: User making changes
317             criteria_updates: New/modified criteria
318
319         Returns:
320             Updated segment with new criteria
321         """
322
323         # Fetch current segment
324         segment = await self.db.fetchrow(
325             "SELECT * FROM segmentations WHERE id = $1",
326             segmentation_id
327         )
328
329         if not segment:
330             raise ValueError(f"Segmentation {segmentation_id} not
331 found")
332
333         # Merge criteria
334         current_criteria = segment["criteria"]
335
336         # Add new filters
337         if "add_filters" in criteria_updates:
338             current_criteria.setdefault("filters", []).extend(
339                 criteria_updates["add_filters"]
340             )
341
342         # Remove filters
343         if "remove_filters" in criteria_updates:
344             current_criteria["filters"] = [
345                 f for f in current_criteria.get("filters", [])

```